

Khóa học lập trình Java

Xử lý JSON bằng Java



Khoa Công nghệ Thông tin
Đại học Công nghiệp Thành phố Hồ Chí Minh

Mục tiêu của buổi học

Giới thiệu về JSON

Cấu trúc JSON

API Java để xử lý JSON



Giới thiệu về JSON

<http://www.json.org/>

- JSON (JavaScript Object Notation) là một giao thức trao đổi dữ liệu nhẹ.
định dạng.
 - o Nó dễ đọc và viết đối với con người.
 - o Nó dễ phân tích và tạo ra dữ liệu đối với máy móc.
 - o Nó dựa trên một tập hợp con của Ngôn ngữ Lập trình JavaScript, Standard. ECMA-262 Phiên bản thứ 3 - Tháng 12 năm 1999.
- JSON là một định dạng văn bản hoàn toàn độc lập với ngôn ngữ lập trình nhưng sử dụng các quy ước quen thuộc với các lập trình viên thuộc họ ngôn ngữ C, bao gồm C, C++, C#, Java, JavaScript, Perl, Python và nhiều ngôn ngữ khác. Những đặc tính này làm cho JSON trở thành một ngôn ngữ trao đổi dữ liệu lý tưởng.
- JSON thường được sử dụng trong các ứng dụng Ajax, cấu hình, cơ sở dữ liệu và các dịch vụ web RESTful. Tất cả các trang web phổ biến đều cung cấp JSON làm định dạng trao đổi dữ liệu với các dịch vụ web RESTful của họ.

Cấu trúc JSON (1)

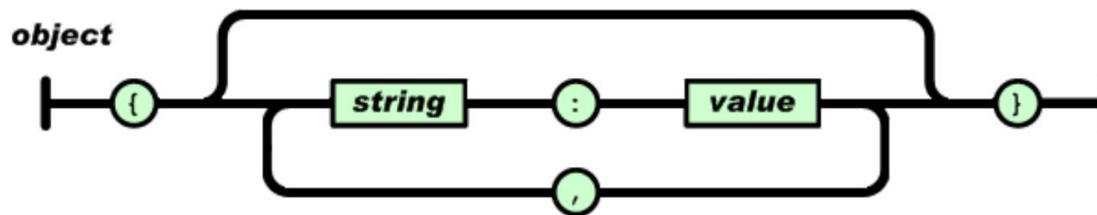
- JSON được xây dựng dựa trên hai cấu trúc:
 - o Một tập hợp các cặp tên/giá trị. Trong các ngôn ngữ khác nhau, điều này được thể hiện dưới dạng đối tượng, bản ghi, cấu trúc, từ điển, bảng băm, danh sách có khóa hoặc mảng liên kết.
 - o Một danh sách các giá trị được sắp xếp. Trong hầu hết các ngôn ngữ lập trình, điều này được thể hiện dưới dạng mảng, vectơ, danh sách hoặc chuỗi.

Cấu trúc JSON (2)

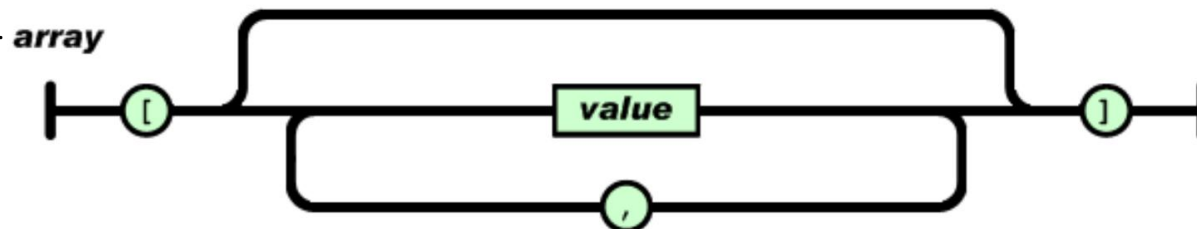
- Trong định dạng JSON, chúng có các dạng sau:

Một **đối tượng** là một tập hợp các cặp tên/giá trị không có thứ tự. Một đối tượng bắt đầu bằng ...

Bắt đầu bằng dấu ngoặc nhọn { (mũi nhọn bên trái) và kết thúc bằng dấu ngoặc nhọn } (mũi nhọn bên phải). Mỗi tên được theo sau bởi dấu hai chấm : (dấu hai chấm) và các cặp tên/giá trị được phân tách bằng dấu phẩy , (dấu phẩy).

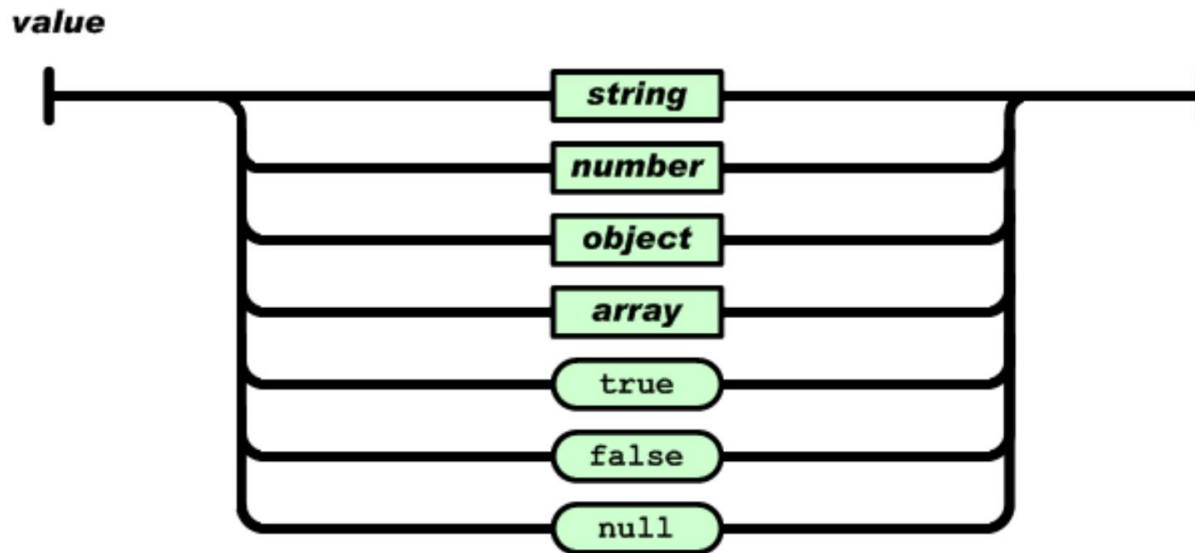


Mảng là một tập hợp các giá trị được sắp xếp theo thứ tự. Mảng bắt đầu bằng dấu ngoặc vuông [(ngoặc vuông trái) và kết thúc bằng dấu ngoặc vuông] (ngoặc vuông phải). Các giá trị được phân cách bởi dấu phẩy , (dấu phẩy).



Cấu trúc JSON (3)

- Một **giá trị** có thể là một chuỗi ký tự trong dấu ngoặc kép, hoặc một số, hoặc true, false, null, hoặc một đối tượng hoặc một mảng. Các cấu trúc này có thể là lồng nhau.

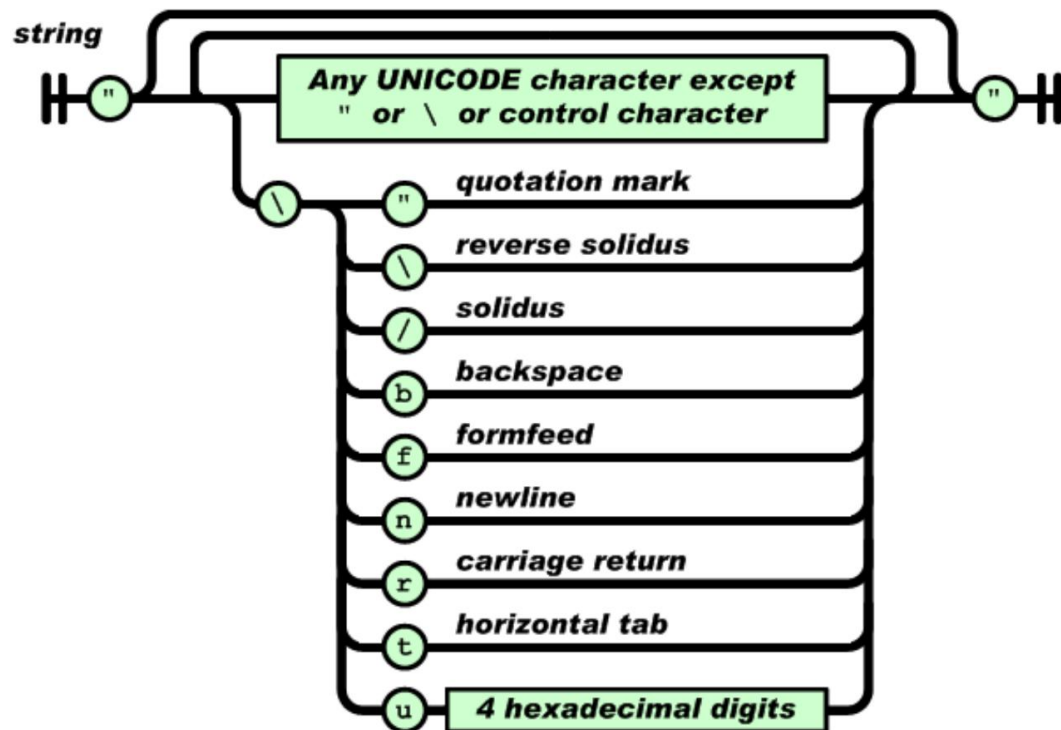


Cấu trúc JSON (4)

- Chuỗi **ký tự** là một dãy gồm không hoặc nhiều ký tự Unicode.

Một ký tự được bao bọc trong dấu ngoặc kép, sử dụng ký tự thoát là dấu gạch chéo ngược.

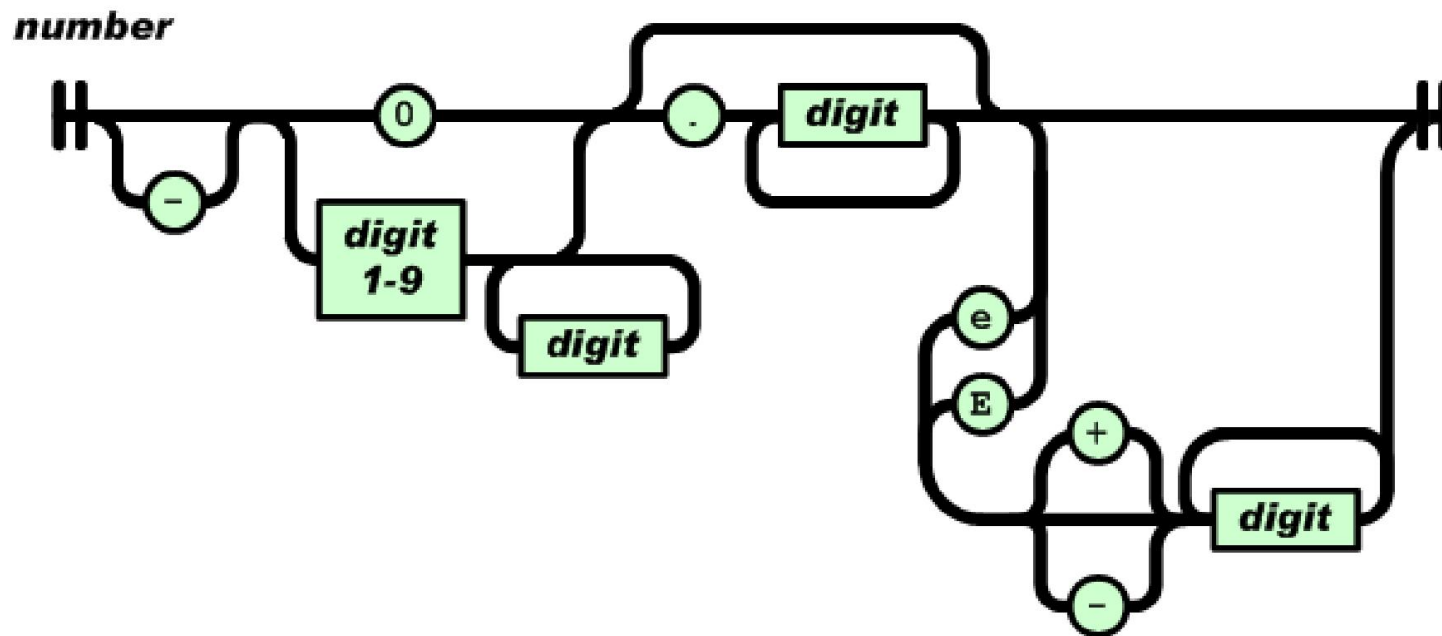
Một ký tự được biểu diễn dưới dạng một chuỗi ký tự đơn. Chuỗi ký tự rất giống với chuỗi ký tự trong C hoặc Java.



Cấu trúc JSON (5)

- Một **số** rất giống với số trong C hoặc Java, ngoại trừ việc...

Các định dạng bát phân và thập lục phân không được sử dụng.



Ví dụ về tài liệu JSON và quy tắc.

```
{ } cust.json
1 {
2   "firstName": "John",
3   "lastName": "Smith",
4   "age": 25,
5   "address": {
6     "streetAddress": "21 2nd Street",
7     "city": "New York",
8     "state": "NY",
9     "postalCode": 10021
10  },
11  "phoneNumbers": [
12    {
13      "type": "home",
14      "number": "212 555-1234"
15    },
16    {
17      "type": "fax",
18      "number": "646 555-4567"
19    }
20  ]
21 }
```

```
object
  {}
  { members }
members
  pair
  pair , members
pair
  string : value
array
  []
  [ elements ]
elements
  value
  value , elements
value
  string
  number
  object
  array
  true
  false
  null
```

API Java để xử lý JSON

- API Java để xử lý JSON ([JSR 353](#)) Cung cấp các API di động để phân tích cú pháp, tạo, chuyển đổi và truy vấn JSON bằng cách sử dụng mô hình đối tượng và API truyền dữ liệu.
- Nó tạo và tiêu thụ văn bản JSON theo kiểu truyền tải dữ liệu liên tục (tương tự như API StAX dành cho XML) và cho phép xây dựng mô hình đối tượng Java cho văn bản JSON bằng cách sử dụng các lớp API (tương tự như API DOM dành cho XML).

Xử lý JSON - API Mô hình Đối tượng (1)

- API Mô hình Đối tượng

- API mô hình đối tượng tạo ra một cấu trúc dạng cây, truy cập ngẫu nhiên, biểu diễn dữ liệu JSON trong bộ nhớ. Sau đó, cây này có thể được điều hướng và truy vấn.

o Mô hình lập trình này linh hoạt nhất và cho phép xử lý các yêu cầu truy cập ngẫu nhiên vào toàn bộ nội dung của cây. Tuy nhiên, nó thường không hiệu quả bằng mô hình xử lý luồng và yêu cầu nhiều bộ nhớ hơn. o API của mô hình đối tượng tương tự như API của Mô hình Đối tượng Tài liệu (DOM).

đối với XML.

- Đó là một API cấp cao cung cấp các mô hình đối tượng bất biến cho cấu trúc đối tượng và mảng JSON. Các cấu trúc JSON này được biểu diễn dưới dạng mô hình đối tượng bằng cách sử dụng các kiểu Java JsonObject và JsonArray.

Xử lý JSON - API Mô hình Đối tượng (2)

- Các lớp và giao diện chính trong API mô hình đối tượng

Mô tả lớp hoặc giao diện	
Json	Chứa các phương thức tĩnh để tạo trình đọc, trình ghi, trình xây dựng JSON và các đối tượng nhà máy của chúng.
JsonGenerator	Ghi dữ liệu JSON vào luồng từng giá trị một.
JsonReader	Đọc dữ liệu JSON từ một luồng và tạo mô hình đối tượng trong bộ nhớ.
JsonObjectBuilder	Tạo mô hình đối tượng hoặc mô hình mảng trong bộ nhớ bằng cách thêm các giá trị từ mã ứng dụng.
JsonWriter	Ghi mô hình đối tượng từ bộ nhớ vào luồng dữ liệu.
JsonValue	Biểu diễn kiểu dữ liệu cho các giá trị trong dữ liệu JSON.
Đối tượng JSON	
Mảng JSON	
Chuỗi JSON	
Số JSON	

Xử lý JSON - API Mô hình Đối tượng (3)

- `JsonObject`, `JsonArray`, `JsonString` và `JsonNumber` là các kiểu con của `JsonValue`. Đây là các hằng số được định nghĩa trong API cho các giá trị JSON là `null`, `true` và `false`.
- API mô hình đối tượng sử dụng các mẫu thiết kế Builder để tạo ra các mô hình đối tượng này từ đầu. Mã ứng dụng có thể sử dụng giao diện `JsonObjectBuilder` để tạo các mô hình đại diện cho các đối tượng JSON. Mô hình kết quả có kiểu `JsonObject`. Mã ứng dụng có thể sử dụng giao diện `JsonArrayBuilder` để tạo các mô hình đại diện cho các mảng JSON. Mô hình kết quả có kiểu `JsonArray`.
- Các mô hình đối tượng này cũng có thể được tạo từ nguồn đầu vào (chẳng hạn như `InputStream` hoặc `Reader`) bằng cách sử dụng giao diện `JsonReader`. Tương tự, các mô hình đối tượng này có thể được ghi vào nguồn đầu ra (chẳng hạn như `OutputStream` hoặc `Writer`) bằng cách sử dụng lớp `JsonWriter`.

Ánh xạ giữa các thực thể JSON và Java

JSON	Java
string	java.lang.String
number	java.lang.Number
true false	java.lang.Boolean
null	null
array	java.util.List
object	java.util.Map

Khi giải mã:

Lớp cụ thể mặc định của `java.util.List` là `org.json.simple.JSONArray`.

Lớp cụ thể mặc định của `java.util.Map` là `org.json.simple.JSONObject`.

Mã hóa JSON trong Java

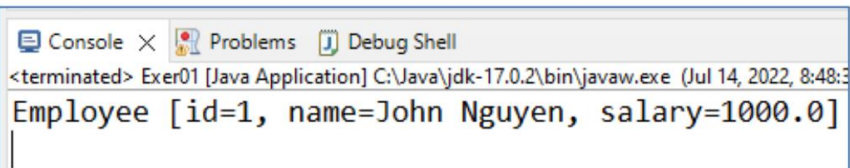
```
public static void main(String[] args) {
    JsonObjectBuilder jsonBuilder = Json.createObjectBuilder();
    Employee employee = new Employee(11, "John Nguyen", 1000);
    JsonObject jo = jsonBuilder
        .add("id", employee.getId())
        .add("name", employee.getName())
        .add("salary", employee.getSalary())
        .build();
    System.out.println(jo.toString());
}
```

```
{
    "id": 1,
    "name": "John Nguyen",
    "salary": 1000.0
}
```

```
<dependencies>
<!-- https://mvnrepository.com/artifact/javax.json/javax.json-api -->
<dependency>
    <groupId>javax.json</groupId>
    <artifactId>javax.json-api</artifactId>
    <version>1.1.4</version>
</dependency>
<!-- https://mvnrepository.com/artifact/org.glassfish/javax.json -->
<dependency>
    <groupId>org.glassfish</groupId>
    <artifactId>javax.json</artifactId>
    <version>1.1.4</version>
</dependency>
</dependencies>
```

Giải mã JSON trong Java

```
public static void main(String[] args) {  
  
    String json = "{\"id\":1,\"name\":\"John Nguyen\",\"salary\":1000.0}";  
    JsonReader reader = Json.createReader(new StringReader(json));  
    JsonObject jo = reader.readObject();  
  
    Employee employee = new Employee(jo.getJsonNumber("id").longValue(),  
                                     jo.getString("name"),  
                                     jo.getJsonNumber("salary").doubleValue());  
    System.out.println(employee);  
}
```



The screenshot shows an IDE window with three tabs: 'Console', 'Problems', and 'Debug Shell'. The 'Console' tab is active, displaying the output of the Java application. The text in the console is: '<terminated> Exer01 [Java Application] C:\Java\jdk-17.0.2\bin\javaw.exe (Jul 14, 2022, 8:48:3' followed by a new line and the output 'Employee [id=1, name=John Nguyen, salary=1000.0]'. The cursor is positioned at the end of the output line.

```
<terminated> Exer01 [Java Application] C:\Java\jdk-17.0.2\bin\javaw.exe (Jul 14, 2022, 8:48:3  
Employee [id=1, name=John Nguyen, salary=1000.0]  
|
```


Xử lý JSON - API truyền phát (1)

- API Truyền phát

API truyền phát dữ liệu cung cấp một cách để phân tích cú pháp và tạo JSON trong một Thời trang trực tuyến.

o Nó chuyển giao quyền kiểm soát việc phân tích cú pháp và tạo dữ liệu cho lập trình viên. o API truyền dữ liệu cung cấp một trình phân tích cú pháp dựa trên sự kiện và cho phép nhà phát triển ứng dụng yêu cầu sự kiện tiếp theo thay vì xử lý sự kiện trong hàm gọi lại. Điều này giúp nhà phát triển kiểm soát quy trình xử lý JSON tốt hơn. Mã ứng dụng có thể xử lý hoặc loại bỏ sự kiện phân tích cú pháp và yêu cầu sự kiện tiếp theo (kéo sự kiện).

Xử lý JSON - API truyền phát (2)

- API truyền dữ liệu trực tuyến tương tự như API truyền dữ liệu trực tuyến dành cho XML.
(StAX) và bao gồm các giao diện JsonParser và
Trình tạo Json.
- JsonParser chứa các phương thức để phân tích dữ liệu JSON bằng cách sử dụng...
mô hình phát trực tuyến.
- JsonGenerator chứa các phương thức để ghi dữ liệu JSON vào nguồn đầu ra. Bảng 2
liệt kê các lớp và giao diện chính trong JsonGenerator.

API truyền phát dữ liệu.

Xử lý JSON - API truyền phát (3)

- Các lớp và giao diện chính trong API truyền phát dữ liệu

Mô tả lớp hoặc giao diện	
Json	Chứa các phương thức tĩnh để tạo trình phân tích cú pháp JSON, trình tạo và các đối tượng nhà máy của chúng.
JsonParser	Đây là một trình phân tích cú pháp dựa trên sự kiện, có khả năng đọc dữ liệu JSON từ một luồng dữ liệu.
JsonGenerator	Ghi dữ liệu JSON vào luồng từng giá trị một.

Xử lý JSON - API truyền phát (3)

- `JsonParser` cung cấp quyền truy cập chỉ đọc, hướng về phía trước vào dữ liệu JSON bằng cách sử dụng mô hình lập trình phân tích cú pháp kéo. Trong mô hình này, mã ứng dụng điều khiển luồng và gọi các phương thức trong giao diện trình phân tích cú pháp để đẩy trình phân tích cú pháp tiến lên hoặc để lấy dữ liệu JSON từ trạng thái hiện tại của trình phân tích cú pháp.
- `JsonGenerator` cung cấp các phương thức để ghi dữ liệu JSON vào một luồng. Trình tạo này có thể được sử dụng để ghi các cặp tên/giá trị trong các đối tượng JSON và các giá trị trong các mảng JSON.
- API truyền dữ liệu là một API cấp thấp được thiết kế để xử lý lượng lớn dữ liệu JSON một cách hiệu quả. Các framework JSON khác (như JSON binding) có thể được triển khai bằng cách sử dụng API này.

Giải mã JSON trong Java - API luồng

```
public static void main(String[] args) {  
    // Parse back  
    final String result = "{\\"name\\":\\"Falco\\",\  
    "age\\":3,\\"bitable\\":false}";  
    final JsonParser parser = Json.createParser(  
new StringReader(result));  
    String key = null;  
    String value = null;  
    while (parser.hasNext()) {  
        final Event event = parser.next();  
        switch (event) {  
            case KEY_NAME:  
                key = parser.getString();  
                System.out.println(key);  
                break;  
            case VALUE_STRING:  
                value = parser.getString();  
                System.out.println(value);  
                break;  
        }  
    }  
    parser.close();  
}
```

Phần kết luận

- API Java để xử lý JSON cung cấp các khả năng sau:
 - o Phân tích luồng đầu vào thành các đối tượng bất biến hoặc luồng sự kiện
 - o Ghi luồng sự kiện hoặc các đối tượng bất biến vào luồng đầu ra
 - o Điều hướng các đối tượng bất biến theo chương trình
 - o Xây dựng các đối tượng bất biến theo chương trình bằng các trình tạo

Câu hỏi thường gặp



Buổi học này đến đây là kết thúc!

Cảm ơn tất cả mọi người đã chú ý và kiên nhẫn!